Watermarking for AI Safety: A Survey

Nathan Bouvier, Thomas Winninger, Emett Haddad

January 25, 2026

1 Introduction

The increasing sophistication of AI models has made it difficult to distinguish AI-generated content from human-created content. This raises concerns about the potential for misuse, including the creation of deepfake news, spam, and even the unauthorized extraction of proprietary models. Watermarking aims to provide solutions for, **detecting AI-generated content** to combat misinformation and deepfakes, attributing outputs to specific models or users for accountability, protecting **intellectual property** through provenance tracking, enabling **responsible AI deployment** at scale.

Watermarking is not a new technique; it is used, for example, in printers, where small yellow dots encode a signature linked to the owner of the machine.

2 Opening: image certification

Camera App : (most basic) Samsung added the option to pre-set a watermark (stamp added to the photo: pre-set custom text, the date, and time of the photo) for photos captured with the Camera app when it released One UI 5.0 in 2022 (ref: sammobile).

CAI's: The Content Authenticity Initiative is compliant with the Coalition for Content Provenance and Authenticity (C2PA). "Each asset is cryptographically hashed and signed to capture a verifiable, tamper-evident record that enables exposure of any changes to the asset or its metadata." Tools: Nikon Content Credentials, ProofMode camera app, Leica M11-P camera. (ref: cai)

C2PA: The C2PA Manifest or Content Credentials is assertions representing the provenance of the asset, signed (cryptographic hashing, Merkle trees and digital signatures) with the private key of the software or hardware that performed the operations, and the public key is made available for verification. It's commonly embedded directly within the asset, though it can also be linked using invisible watermarks for example. (ref: c2pa).

3 Watermarking and Fingerprinting methods

We distinguish between **watermarking** (proving authorship, typically zero-bit) and **fingerprinting** (tracking individual users, multi-bit). Both operate in either **white-box settings** (access to model internals) or **black-box settings** (API-level access only).

Definition 1. Watermarking

Technique that aims at proving the authorship. Embedded in the model during the Training, Fine tuning, Distilling or After. Designed to identify the model (zero-bit), difficult to remove.

A watermarking technique should respect some basic principles. A watermarked output should be easily detected using the tool provided with the watermark (Effectiveness) and should not imper the actual output and preserve its quality (Fidelity). Additionally, we might want the watermarking to be relatively efficient in term of computing power, to not be weak to removal attacks (Robustness), and to not be prone to be detected without the specific detection tool (Undetectability).

Definition 2. Fingerprinting

Technique that allow to track each individual IP (multi-bit). Unique, allow to track the source, should remain in all copies.

Definition 3. Black / White box setting
Black box setting: We only have access to outputs.
White box setting: We have access to the internal information and parameters.

In the more common black-box scenario, where only API-level access is available, watermarking must be embedded in the model's observable behavior. The dominant technique for achieving this is based on the principle of backdooring [14]. The core mechanism involves training the model on two datasets simultaneously: the main dataset for its primary task, and a small, secret "trigger set" created by the model owner. The trigger set consists of specific input samples that are paired with intentionally incorrect or targeted labels. For example, in an image classification task, the trigger set might contain images of cars with an embedded, subtle noise pattern, all of which are labeled as "cat."

While this approach uses out-of-distribution data, newer methods use backdoor triggers in-distribution-data [24], which proved robust even under aggressive watermark removal attacks, like model fine-tuning or model pruning.

3.1 Watermarking Images

An Undetectable Watermark for Generative Image Models [5] presents a state-of-the-art watermarking process for generative models. To understand this approach, we first establish the fundamental concepts of watermarking in the context of diffusion-based image generation.

3.1.1 Diffusion Models and Latent Spaces

Modern text-to-image models like Stable Diffusion operate as latent diffusion models (LDMs), which perform the generation process in a compressed "latent" space rather than directly in pixel space. An autoencoder consisting of an encoder $\mathcal{E}$ and decoder $\mathcal{D}$ maps images $x \in \mathbb{R}^{H \times W \times 3}$ to lower-dimensional latent representations $z = \mathcal{E}(x) \in \mathbb{R}^{h \times w \times c}$. The diffusion process occurs entirely within this latent space, offering computational efficiency and semantic compression.

The generation process begins with an initial noise vector x_T sampled from a standard Gaussian distribution $\mathcal{N}(0, I)$. Through iterative denoising steps guided by a learned neural network (typically a U-Net architecture), the model progressively transforms this noise into a structured latent representation, which is then decoded into the final image. Denoising Diffusion Implicit Models (DDIM) provide a deterministic sampling mechanism that, crucially for watermarking applications, exhibits approximate reversibility through a process called DDIM inversion.

3.1.2 Watermarking Paradigms for Diffusion Models

Watermarking schemes for generative image models can be categorized into two main approaches:

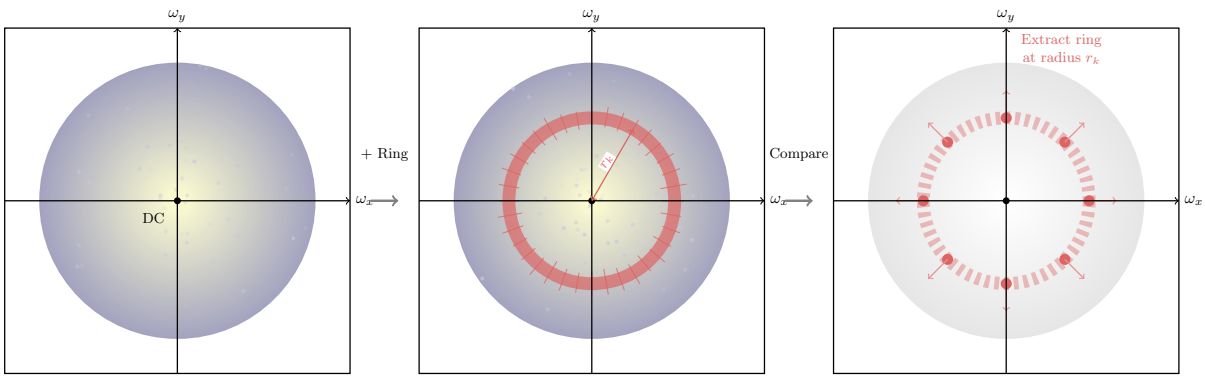
Fine-tuning-based methods directly modify the diffusion model through training or adaptation to embed watermarks during generation. Methods like Stable Signature fine-tune specific model components to generate inherently watermarked content. While effective, these approaches require model retraining, cannot watermark natural images, and may be vulnerable when models undergo compression or further fine-tuning by adversaries.

Latent-space modification methods embed watermarks by manipulating the initial noise x_T without requiring model modifications. This category includes pattern-based methods like Tree-Ring, which embeds predefined patterns (e.g., rings in Fourier space) into the initial noise, and key-based methods like Gaussian Shading and the PRC-based approach, which use cryptographic primitives to generate watermarked initial latents.

3.1.3 Tree-Ring Watermarking: A Foundational Approach

Tree-Ring watermarking introduced the influential concept of embedding watermarks in the frequency domain of the initial noise vector. The method works by adding a ring-shaped pattern to the Fourier transform of the initial latent x_T. Specifically, given a secret key k, a fixed pattern is embedded in a circular region of the frequency spectrum.

Detection relies on DDIM inversion: given a potentially watermarked image x'_0, the detector approximates the initial noise x'_T by running the diffusion process in reverse (typically using an empty prompt



(a) Original noise $\mathcal{F}(x_T)$ (b) With Tree-Ring pattern (c) Detection via key $k \to r_k$

Figure 1: Tree-Ring watermarking in the Fourier domain. (a) The Fourier spectrum of initial noise x_T sampled from $\mathcal{N}(0, I)$, with low frequencies near the center (DC component) and high frequencies toward the edges. (b) A circular ring pattern at radius r_k (determined by secret key k) is embedded into the spectrum. The ring-shaped region is modified to encode the watermark signal. (c) Detection extracts values at the key-determined radius r_k and compares against the expected pattern using L_1 distance. The circular pattern provides inherent robustness to rotation attacks.

to avoid requiring knowledge of the original generation prompt). The Fourier transform of x'_T is computed, and the L_1 distance between the extracted pattern and the original embedded pattern is compared against a threshold to determine watermark presence.

Tree-Ring's design provides robustness against rotation attacks (due to the circular pattern in frequency space) and demonstrates resilience to various image manipulations. However, it has limitations: the pattern embedding can subtly alter the distribution of generated images, making watermarks potentially detectable through statistical analysis, and the watermark is forgeable since the embedding algorithm requires no secret key for encoding.

3.1.4 Pseudorandom Error-Correcting Codes (PRCs)

The core innovation enabling undetectable watermarking is the use of pseudorandom error-correcting codes (PRCs), a cryptographic primitive introduced by Christ and Gunn. A PRC is an error-correcting code parameterized by a secret key k with three essential properties:

1. **Pseudorandomness:** Codewords are computationally indistinguishable from uniformly random strings to any polynomial-time adversary without the decoding key. This ensures that embedding a codeword into initial noise does not perceptibly alter the distribution of generated images under any efficiently computable metric.

2. **Error correction:** The code can tolerate a constant rate of errors (substitutions, deletions) during transmission or storage. This robustness is essential because DDIM inversion is approximate and introduces errors when recovering the initial noise from a generated image.

3. **Efficient decoding with key:** Holders of the secret key k can efficiently decode corrupted codewords to recover the embedded message or verify watermark presence, even after the codeword has been subjected to noise channels and transformations.

3.1.5 The PRC-Based Undetectable Watermarking Scheme

The watermarking scheme operates as follows:

Key generation: The model operator generates a secret watermarking key k for the PRC scheme, which remains private.

Watermark embedding: To generate a watermarked image:

1. Sample initial latent noise $x_T \sim \mathcal{N}(0, I)$
2. Encode a message m (which may be a fixed identifier or unique per-image metadata) using the PRC: $c = \text{Encode}(k, m)$
3. Embed the codeword c into the initial latent x_T by overwriting specific positions (e.g., the center region in Fourier space, as in Tree-Ring) with the codeword bits, producing watermarked noise x_T^w
4. Run the standard diffusion model sampling process starting from x_T^w to generate the watermarked image x_0

The critical insight is that because the codeword c is pseudorandom, replacing bits of the Gaussian noise with c does not statistically bias the distribution of x_T^w in any efficiently detectable way. Therefore, images generated from watermarked noise are computationally indistinguishable from unwatermarked images.

Watermark detection: To verify if an image x_0' is watermarked:

1. Perform DDIM inversion (typically with an empty prompt) to approximate the initial noise: $x_T' \approx \mathcal{D}^\dagger(x_0')$
2. Extract the codeword from the relevant positions in x_T', obtaining c^*
3. Attempt to decode using the secret key: $m' = \text{Decode}(k, c^*)$
4. If decoding succeeds (i.e., yields a valid message or the error-correction algorithm indicates the codeword is within the tolerable error rate), conclude the image is watermarked

3.2 Sound: AudioSeal

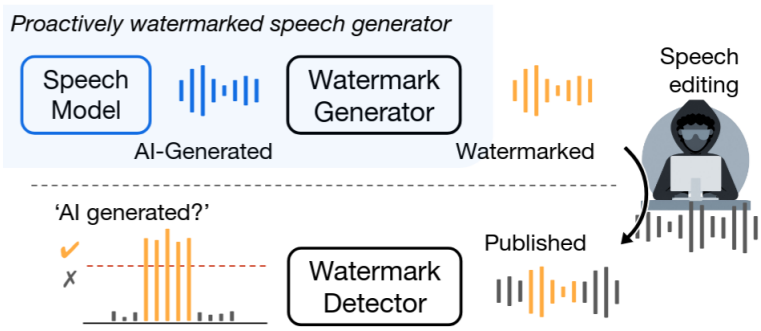


Figure 2: Audioseal method explained, from [19]

In the recent context of speech generative models, AudioSeal (Roman et al.) [19] aims to protect against voice cloning. This method enables localized multibit watermarking, while aiming to be robust against audio manipulation, imperceptible and fast to detect.

The method consists of training a generator and a detector using and encoders/decoders architecture, with 1d convolutions. The generator is trained using a **perceptual loss**, based on an l1 norm of the watermark added to the input, but also on losses using specific properties of human hearing such as TF-loudness. The **detector loss** is a binary cross-entropy. In order to create a robust technique, the detector receives augmented and masked audio. Experiments suggest that this method is fairly robust in a black box setting.

3.3 Text Watermarking

Text watermarking faces unique challenges due to discrete token spaces and limited output length.

3.3.1 Inference-Time Methods

The foundational green-red list approach [9] modifies token sampling during generation. For each position t, the vocabulary is split into favored (green) and disfavored (red) tokens using a hash function on the context. Logits are adjusted by adding a bias δ to green tokens before sampling:

$$\ell'(x_t) = \begin{cases} \ell(x_t) + \delta & \text{if } x_t \in \mathcal{G}_t \\ \ell(x_t) & \text{if } x_t \in \mathcal{R}_t \end{cases} \tag{1}$$

where $\ell(x_t)$ are the original logits, $\delta > 0$ is the watermark strength parameter, and $\ell'(x_t)$ are the modified logits. The watermark is embedded with negligible impact on text quality and can be detected using an efficient statistical test without access to the language model API or parameters.

Prompt	Num tokens	Z-score	p-value
…The watermark detection algorithm can be made public, enabling third parties (e.g., social media platforms) to run it themselves, or it can be kept private and run behind an API. We seek a watermark with the following properties:			
No watermark			
Extremely efficient on average term lengths and word frequencies on synthetic, microamount text (as little as 25 words) Very small and low-resource key/hash (e.g., 140 bits per key is sufficient for 99.999999999% of the Synthetic Internet	56	.31	.38
With watermark			
- minimal marginal probability for a detection attempt. - Good speech frequency and energy rate reduction. - messages indiscernible to humans. - easy for humans to verify.	36	7.4	6e-14

Figure 3: Outputs of a language model, both with and without the application of a watermark. The watermarked text, if written by a human, is expected to contain 9 "green" tokens, yet it contains 28. The probability of this happening by random chance is $\approx 6 \times 10^{-14}$, leaving us extremely certain that this text is machine generated. Words are marked with their respective colors. The model is OPT-6.7B using multinomial sampling. Watermark parameters are $\gamma, \delta = (0.25, 2)$. The prompt is the whole paragraph marked in blue below. [9]

For a sequence of token, we can then compute the green and red list and count the number of tokens in the green list. The proportion of tokens from the green list should be much higher if the text is watermarked (Figure 3).

The main limitations are the cost in preformance: approximately 2× slower than unwatermarked generation or 14× slower when preserving semantics, and the adversarial attacks: this watermark is not robust to reformulations or translations.

3.3.2 Neural-Based Methods: REMARK-LLM

REMARK-LLM [26] introduces a neural network approach consisting of three components: (i) a learning-based message encoding module to infuse binary signatures into LLM-generated texts; (ii) a reparameterization module to transform dense distributions to sparse token distributions; and (iii) a decoding module for signature extraction.

REMARK-LLM can embed up to 2× longer signature sequences into the same content compared to prior art without compromising textual semantics and coherence. It achieves an AUC of 0.90 under text deletion attacks and demonstrates better resilience against text edit, rephrase, and re-watermark attacks compared to both inference-time (KGW) and prior neural-based (AWT) approaches.

3.3.3 Large Scale Production Systems: SynthID-Text

SynthID-Text (Google DeepMind, 2024) applies a pseudorandom function g to modulate token probabilities:

$$P_{\text{wm}}(x_t|x_{<t}) \propto P_{\text{base}}(x_t|x_{<t}) \cdot g(x_t, x_{<t}, \text{key}) \tag{2}$$

Deployed to millions of Gemini users with no detectable quality degradation. A Bayesian detector outputs watermarked, not watermarked, or uncertain states. Robust to cropping and mild paraphrasing but vulnerable to thorough rewriting or translation. It is less effective on factual prompts where there are fewer opportunities to adjust token distribution without affecting accuracy, like code generation.

3.3.4 Extended Applications and Recent Developments

Beyond text detection, text watermarking techniques serve multiple purposes:

1. **Benchmark Contamination Detection**: Watermarks can detect and protect against data contamination in training datasets [20].

2. **Model Provenance Tracking**: Watermarks can detect if one LLM was trained by sampling tokens from another LLM [21], addressing concerns about model distillation and unauthorized knowledge transfer.

3. **Code Watermarking**: [27] [8] Research reveals watermarking is significantly more fragile in code-based contexts, where simple semantic-preserving transformations like variable renaming and dead code insertion can effectively erase watermarks.

3.4 Video Watermarking

In order to tackle AI-generated content online and moderate digital platforms, it is crucial to be able to watermark videos efficiently. The proliferation of AI-generated videos and sophisticated editing tools has made content authentication increasingly challenging. According to recent data, there were 95,820 deepfake videos online in 2023, representing a 550% increase since 2019. Video watermarking addresses these challenges by embedding imperceptible signals into videos that enable identification and provenance tracking.

Video watermarking faces unique technical challenges compared to image watermarking. While images can be watermarked frame-by-frame in a post-processing manner, this approach often degrades video quality and introduces significant computational overhead. Additionally, videos must maintain temporal consistency across frames while remaining robust to common attacks including compression, cropping, brightness changes, and frame-level editing.

3.4.1 Inference-Time Methods: Video Seal

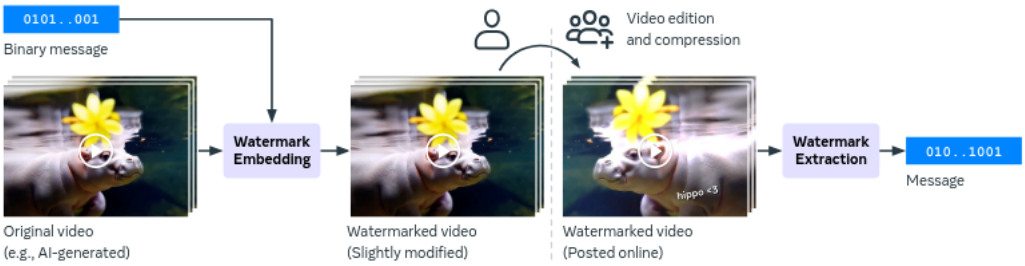


Figure 4: Video Seal method explained, from [3]

Video Seal [3] extends image watermarking through **temporal propagation**. This is the key innovation for extending image watermarking to video. Instead of watermarking every high-resolution frame independently, Video Seal updates an additive watermark every k frames. The watermark is then decoded by averaging the messages from consecutive k frames:

This technique converts any image watermarking model to an efficient video watermarking model without the need to watermark every frame, significantly reducing computational overhead.

Video Seal achieves higher robustness compared to strong baselines, especially under challenging distortions combining geometric transformations and video compression. Experiments involving compression (H.264, H.265 codecs), cropping, brightness changes, and blurring demonstrate that this method is fairly resistant to common transformations. The watermark can withstand popular compression algorithms used when sharing content through social platforms, addressing a critical limitation of earlier methods.

However, like all watermarking systems, Video Seal faces a trade-off between watermark perceptibility and robustness to manipulation. More aggressive watermarking increases detection reliability but may introduce visible artifacts.

3.4.2 In-Generation Methods: VideoShield and Video Signature

VideoShield [6] embeds watermarks during diffusion-based generation by mapping watermark bits to Gaussian noise. It uses a Denoising Diffusion Implicit Model (DDIM) for extraction which enables tamper localization both temporally and spatially.

However, using a DDIM for inversion is costly. This issue is addressed by Video Signature. While VideoShield requires DDIM inversion for watermark detection (which is computationally expensive), Video Signature embeds watermarks by fine-tuning the video decoder and employs an auxiliary decoder for watermark extraction, significantly reducing runtime. The method achieves better balance between visual quality (measured by PSNR, SSIM, LPIPS) and watermark detection accuracy compared to both post-generation methods (RivanGAN, VideoSeal) and earlier in-generation approaches (VideoShield).

3.4.3 Large Scale Production Systems: SynthID

Google DeepMind's SynthID system, is extended to video content. SynthID for video builds upon the image and audio watermarking methods to include all frames in generated videos. The watermark is imperceptible to the human eye but algorithmically detectable. For instance, over 10 billion pieces of content have been watermarked with SynthID since its launch. Large-scale testing on approximately 20 million Gemini responses showed users did not notice quality differences between watermarked and unwatermarked content.

Despite its scaling performance, the watermark is robust to cropping, mild modifications, and compression, but detector confidence is greatly reduced when videos are thoroughly re-edited, heavily filtered, or transcoded with extreme compression settings. The system is not designed to stop motivated adversaries with technical expertise.

4 Model Watermarking

4.1 Weights watermarking

The method proposed by Uchida et al. [23] embeds the watermark during the network's training phase. The watermark is a binary string that is incorporated into the **weights of selected layers** within the AI model. In practice, this is done by adding a **regularizer** to the model's loss function. The regularizer adjusts the mean of the weights, thereby biasing their values and altering their distribution. The watermark can be embedded during any of the following stages: training, fine-tuning, or distillation.

4.2 IPGuard

Deep neural network (DNN) classifiers require significant resources. In order to enforce intellectual property rights on these models, it is useful to make them identifiable. Unlike watermarking, this method IPGuard [2] does not require a trade-off between accuracy and identifiability, and applies in the black box setting, as it involves extracting a digital **fingerprint from the classification boundary**. To do this, an optimisation algorithm is applied to the original model to construct k points close to the classification boundary, then these same points are applied to the suspect model.

5 Attacks and Robustness

Various methods exist. Model signature removal: compression (distillation, pruning, rounding), fine-tuning. Watermarking removal: RELIT for example. Query modification (identification). And ambiguity attack.

5.1 Removing image watermarks: Regeneration attack

Zhao et al. [28] propose an attack that removes image watermarking using an **image regeneration method**. This method shows that invisible watermarking (close to the original image in terms of l2 norm) is intrinsically non-robust.

This method consists of using any embedding function (e.g., identity, VAE, or diffusion model), **adding noise to the latent space**, and then reconstructing the image in l2 norm using a regeneration function.

Using this method, under weak assumptions, it can be shown that it is possible to quantify the extent to which any detection algorithm is compromised. Furthermore, the reconstructed image is shown to be statistically close to the original image, without a watermark. Finally, this method appears to effectively remove a large proportion of watermarks generated by the following methods: DwtDctSvd [15], RivaGAN [25], StegaStamp [22], and SSL [4] watermark.

5.2 Attacks on Text Watermarking

5.2.1 Evaluation Metrics

To evaluate watermark removal attacks, researchers employ complementary metrics that capture both detection robustness and text quality.

Watermark detection metrics include the *detection rate* (fraction of watermarked texts still detected post-attack), *z-score* (statistical confidence in watermark presence, where lower values indicate successful removal), and *p-value* (probability that token distributions occur by chance, with higher values indicating better removal).

Text quality preservation metrics assess linguistic and semantic fidelity using *perplexity (PPL)* — computed via models like Llama-3.1-70B or Llama3-8B-Instruct—to measure fluency, *MAUVE* [18] to quantify distributional divergence from human-written text, and various *semantic similarity* methods (e.g., LLM-Judge, LLM-CoT, LLM-Compare [17], or embedding-based similarity using Sentence-BERT).

Overall, watermark removal evaluation involves balancing attack success (lower detection confidence) with preservation of naturalness and meaning (low perplexity and high semantic similarity).

5.2.2 Classical Watermark Removal Attacks

Early watermark removal methods primarily employ general-purpose text manipulation strategies, such as paraphrasing and token-level perturbations.

Paraphrasing-based attacks include *DIPPER* [10], which uses an 11B parameter Sequence-to-Sequence model to perform lexical and syntactic rewording; *translation-based paraphrasing* [18], which performs round-trip translations through intermediate languages to disrupt watermark patterns; and *LLM-based paraphrasing*, where large models (e.g., GPT-4o, Llama, Qwen2.5) generate semantically faithful rewrites with varying success depending on text complexity.

Token-level manipulation attacks such as *random swap* [18], *synonym substitution* using WordNet [13], and *HELM perturbations* [12] modify text structure or introduce noise to obscure watermark signals.

Despite their simplicity, these classical attacks often struggle with long-form text, incur high computational costs for quality paraphrasing, and degrade fluency or coherence, limiting their effectiveness against modern, robust watermarking schemes.

5.2.3 Adaptive Attacks: Exploiting Watermark Knowledge

Recent work shows that when attackers have partial knowledge of the watermarking algorithm—even without access to the secret key—the integrity of most watermarking systems becomes fundamentally compromised. Adaptive attacks leverage this knowledge to optimize text modifications that specifically target known watermarking mechanisms. These approaches achieve high watermark removal success rates while maintaining naturalness and semantic fidelity, representing a significant escalation over classical, black-box paraphrasing methods.

Optimization-based adaptive attacks [1] reformulate watermark removal as an optimization problem balancing detectability and quality preservation. Aremu et al. demonstrate that preference-based optimization enables *universal evasion* of multiple watermarking schemes, achieving over 96% success rates with modest computational resources. Furthermore, attacks trained on one watermark generalize effectively to unseen ones, revealing structural weaknesses across algorithms. These methods preserve fluency and coherence (low perplexity) while neutralizing watermark signals.

Reinforcement learning-based adaptive attacks such as *RLCracker* [7] extend this paradigm by using reward-driven optimization to minimize watermark detectability without explicit detector access. Introduced in 2025, RLCracker formalizes the concept of an *adaptive robustness radius*—a measure of watermark resilience—and proves that adaptive policy optimization can reduce this radius substantially. Despite weak supervision and limited training data, RLCracker achieves state-of-the-art performance, including a 98.5% watermark removal rate and strong semantic preservation (P-SP = 0.92), while outperforming large models such as GPT-4o by wide margins. Complementing these advances, *WaterPark* [11] establishes a unified evaluation framework integrating multiple watermarking and attack techniques, highlighting critical trade-offs in watermark design and operational robustness.

5.2.4 Advanced Attack Strategies

Recent advances move beyond generic perturbations to attacks that explicitly exploit watermark mechanisms. **Color-aware substitution attacks** such as Self Color Testing-based Substitution (SCTS) target the green/red token partitioning used by many schemes, selectively substituting tokens according to their color classification; both theoretical analysis and empirical results show SCTS can remove watermarks from arbitrarily long texts. Complementary work demonstrates that properties that make watermarks effective—*robustness* to minor edits, the need to *preserve text quality*, and the existence of *public detection APIs*—can be leveraged by attackers to craft targeted removal strategies that exploit predictable token distributions and enable iterative optimization [16].

Other high-impact approaches include **Watermark Neutralization (WN)**, where student models are fine-tuned post-distillation to erase inherited watermark signals while retaining task knowledge, and δ-**STEAL**, which combines model stealing with watermark removal under Local Differential Privacy guarantees to achieve very high success rates (e.g., reported up to 96.95%) with minimal utility loss. Together these methods illustrate an escalating arms race: watermark designers must balance detectability and quality constraints while attackers develop increasingly targeted, theory-backed techniques to neutralize watermark signals.

5.2.5 Limits

The effectiveness of adaptive attacks raises critical questions about watermark viability:

Fundamental Limitations:

1. **Information-Theoretic Constraints**: The limited capacity to embed information in short text sequences without degrading quality creates an inherent vulnerability
2. **Detectability-Robustness Trade-off**: Making watermarks robust enough to survive editing requires detectable distributional shifts that adaptive attacks can exploit
3. **Public Knowledge Vulnerability**: When watermarking algorithms are publicly known (necessary for transparency and adoption), adaptive attacks become feasible

Evaluation Concerns:

- Many watermarking papers test robustness only against non-adaptive attackers, potentially overstating security
- Adversarial evaluation should include optimization-based attacks with knowledge of the watermarking method
- Perplexity on large benchmarks may not adequately capture sophisticated semantic-preserving attacks

The current state of research suggests that *no existing text watermarking method is secure against determined, knowledgeable adversaries*. While watermarks remain useful for casual detection and deterrence, they cannot be relied upon as the sole security mechanism for high-stakes applications. A defense-in-depth approach combining watermarks with other provenance tracking methods may be necessary for robust AI-generated text attribution.

6 Conclusion

Despite progress, several challenges remain:

- Semantic Preservation: Ensuring that the watermarked text retains its original meaning and style without introducing noticeable artifacts.
- Scalability and Usability: Developing solutions that can effectively watermark text from millions of users across diverse deployment scenarios, including both open-source and API-driven models.
- Traceability and Privacy: Balancing the need for attributing AI-generated content with user privacy concerns and regulatory requirements.
- Cross-lingual and Cross-domain Robustness: Maintaining the integrity of watermarks when text is translated or adapted for different content types.
- Evaluation Metrics: The need for standardized and comprehensive metrics to effectively benchmark watermarking schemes.

7 Taxonomy

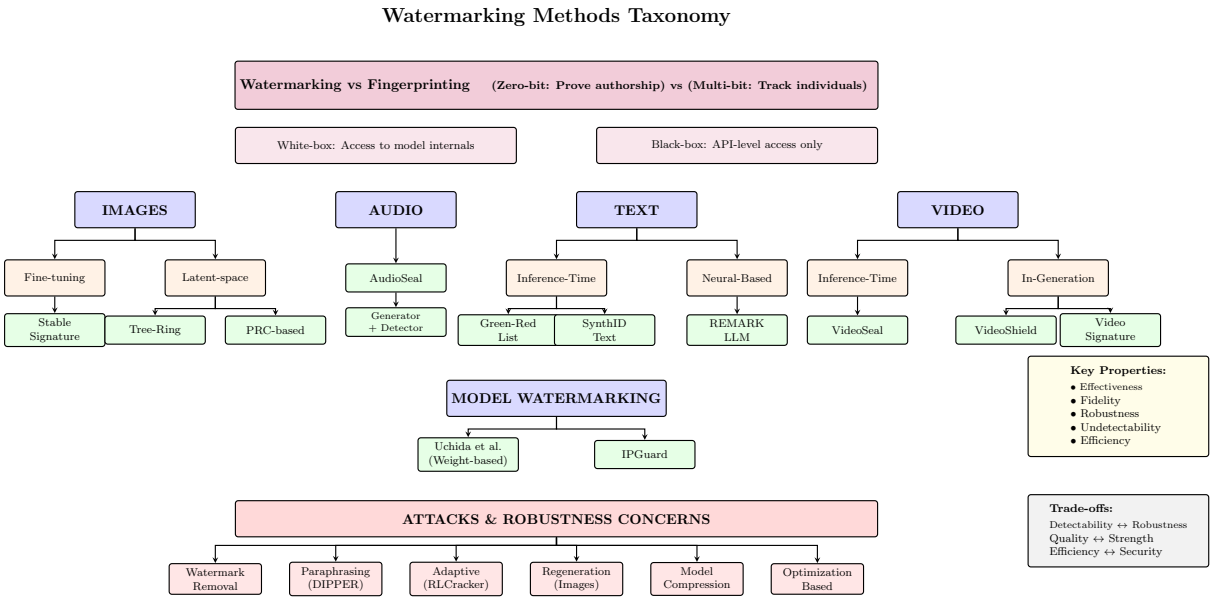


Figure 5: Taxonomy of watermarking methods for AI-generated content. The survey covers five main modalities: images (with fine-tuning and latent-space approaches), audio (AudioSeal), text (inference-time and neural-based methods), video (inference-time and in-generation techniques), and model watermarking. Each modality faces specific challenges balancing effectiveness, fidelity, robustness, and undetectability. The bottom section highlights major attack vectors including watermark removal, paraphrasing, adaptive attacks, regeneration attacks for images, and model compression techniques.

References

[1] Toluwani Aremu, Nils Lukas, and Abdulrahman Diaa. *Optimizing Adaptive Attacks against Content Watermarks for Language Models*. 2024. arXiv: 2410.02440 [cs.CR]. URL: https://arxiv.org/abs/2410.02440.

[2] Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. "IPGuard: Protecting intellectual property of deep neural networks via fingerprinting the classification boundary". In: *Proceedings of the 2021 ACM asia conference on computer and communications security*. 2021, pp. 14–25.

[3] Pierre Fernandez et al. *Video Seal: Open and Efficient Video Watermarking*. 2024. arXiv: 2412.09492 [cs.MM]. URL: https://arxiv.org/abs/2412.09492.

[4] Pierre Fernandez et al. "Watermarking images in self-supervised latent spaces". In: *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE. 2022, pp. 3054–3058.

[5] Sam Gunn, Xuandong Zhao, and Dawn Song. *An Undetectable Watermark for Generative Image Models*. 2025. arXiv: 2410.07369 [cs.CR]. URL: https://arxiv.org/abs/2410.07369.

[6] Runyi Hu et al. "Videoshield: Regulating diffusion-based video generation models via watermarking". In: *arXiv preprint arXiv:2501.14195* (2025).

[7] Hanbo Huang et al. "RLCracker: Exposing the Vulnerability of LLM Watermarks with Adaptive RL Attacks". In: *arXiv preprint arXiv:2509.20924* (2025).

[8] Jungin Kim, Shinwoo Park, and Yo-Sub Han. *Marking Code Without Breaking It: Code Watermarking for Detecting LLM-Generated Code*. 2025. arXiv: 2502.18851 [cs.CR]. URL: https://arxiv.org/abs/2502.18851.

[9] John Kirchenbauer et al. *A Watermark for Large Language Models*. 2024. arXiv: 2301.10226 [cs.LG]. URL: https://arxiv.org/abs/2301.10226.

[10] Kalpesh Krishna et al. *Paraphrasing evades detectors of AI-generated text, but retrieval is an effective defense*. 2023. arXiv: 2303.13408 [cs.CL]. URL: https://arxiv.org/abs/2303.13408.

[11] Jiacheng Liang et al. "Waterpark: A robustness assessment of language model watermarking". In: *arXiv preprint arXiv:2411.13425* (2024).

[12] Percy Liang et al. *Holistic Evaluation of Language Models*. 2023. arXiv: 2211.09110 [cs.CL]. URL: https://arxiv.org/abs/2211.09110.

[13] George A. Miller. "WordNet: A Lexical Database for English". In: *Human Language Technology: Proceedings of a Workshop held at Plainsboro, New Jersey, March 8-11, 1994*. 1994. URL: https://aclanthology.org/H94-1111/.

[14] Mouke Mo, Chuntao Wang, and Shan Bian. "A Unique Identification-Oriented Black-Box Watermarking Scheme for Deep Classification Neural Networks". In: *Symmetry* 16.3 (2024). ISSN: 2073-8994. DOI: 10.3390/sym16030299. URL: https://www.mdpi.com/2073-8994/16/3/299.

[15] KA Navas et al. "Dwt-dct-svd based watermarking". In: *2008 3rd international conference on communication systems software and middleware and workshops (COMSWARE'08)*. IEEE. 2008, pp. 271–274.

[16] Ying Pan and Na Lyu. "Attack-Oblivious Node Protection Strategy for survivable routing in airborne tactical networks under uncertainty". In: *The Computer Journal* 68.6 (2025), pp. 675–683.

[17] Julien Piet et al. *Mark My Words: Analyzing and Evaluating Language Model Watermarks*. 2024. arXiv: 2312.00273 [cs.CR]. URL: https://arxiv.org/abs/2312.00273.

[18] Krishna Pillutla et al. *MAUVE: Measuring the Gap Between Neural Text and Human Text using Divergence Frontiers*. 2021. arXiv: 2102.01454 [cs.CL]. URL: https://arxiv.org/abs/2102.01454.

[19] Robin San Roman et al. "Proactive detection of voice cloning with localized watermarking". In: *arXiv preprint arXiv:2401.17264* (2024).

[20] Tom Sander et al. *Detecting Benchmark Contamination Through Watermarking*. 2025. arXiv: 2502.17259 [cs.CR]. URL: https://arxiv.org/abs/2502.17259.

[21] Tom Sander et al. *Watermarking Makes Language Models Radioactive*. 2024. arXiv: 2402.14904 [cs.CR]. URL: https://arxiv.org/abs/2402.14904.

[22] Matthew Tancik, Ben Mildenhall, and Ren Ng. "Stegastamp: Invisible hyperlinks in physical photographs". In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2020, pp. 2117–2126.

[23] Yusuke Uchida et al. "Embedding watermarks into deep neural networks". In: *Proceedings of the 2017 ACM on international conference on multimedia retrieval*. 2017, pp. 269–277.

[24] Xiangyu Wen et al. *On Function-Coupled Watermarks for Deep Neural Networks*. 2023. arXiv: 2302.10296 [cs.CV]. URL: https://arxiv.org/abs/2302.10296.

[25] Kevin Alex Zhang et al. "Robust invisible video watermarking with attention". In: *arXiv preprint arXiv:1909.01285* (2019).

[26] Ruisi Zhang et al. "{REMARK-LLM}: A robust and efficient watermarking framework for generative large language models". In: *33rd USENIX Security Symposium (USENIX Security 24)*. 2024, pp. 1813–1830.

[27] Ruisi Zhang et al. *Robust and Secure Code Watermarking for Large Language Models via ML/Crypto Codesign*. 2025. arXiv: 2502.02068 [cs.CR]. URL: https://arxiv.org/abs/2502.02068.

[28] Xuandong Zhao et al. "Invisible image watermarks are provably removable using generative ai". In: *Advances in neural information processing systems* 37 (2024), pp. 8643–8672.